MODUL PRAKTIKUM 5

DATABASE OPTIMIZATION

Lanjutan Project Minggu 4 + PostgreSQL + Profiling

Pemrograman Sisi Server — A11.64707

Suprayogi, M.Kom

Informasi	Keterangan
Pertemuan	Minggu 5
Sub-CPMK	Mampu mengoptimalkan performa database dan query untuk aplikasi backend
Metode	Konsep + Demo Dosen + Guided Lab + Profiling + Before/After Benchmark + Tugas
Dataset	Minimal 10.000 records untuk pengujian performa
Environment	Source code di host/VS Code; Django + PostgreSQL melalui Docker Compose
Capstone	Simple LMS — Milestone 5: Database & Performance Optimization
Tugas	Tugas Minggu 5 — Performance Optimization Report

PRINSIP MINGGU 5

Optimasi harus berdasarkan pengukuran. Setiap perubahan harus menjawab: masalah apa yang ditemukan, metrik awalnya berapa, teknik apa yang diterapkan, dan apakah hasilnya benar-benar membaik?

1. Posisi Praktikum dalam RPS

Minggu 5 merupakan eksperimen performa terhadap Django models yang telah dibangun pada Minggu 4. Praktikum meneruskan repository dan environment Docker yang sama; mahasiswa tidak membuat project Django baru.

Tahap	Fokus
Minggu 4	Django + ORM + PostgreSQL + Core LMS Models + Assignment/Submission
Minggu 5	Profiling + N+1 + ORM optimization + EXPLAIN + Index +Benchmark
Minggu 6	REST API Django Ninja dibangun di atas query yang sudah dianalisis

Alur kerja: ukur → temukan bottleneck → buat hipotesis → optimalkan → ukur ulang → analisis trade-off.

2. Capaian Praktikum

- Menjelaskan query count, execution time, N+1 query, index, dan caching.
- Menggunakan profiling melalui CaptureQueriesContext dan/atau Django Debug Toolbar.
- Membangkitkan dataset pengujian minimal 10.000 record secara repeatable di environment Docker.
- Mengidentifikasi N+1 pada ForeignKey dan reverse/many-valued relationship.
- Menggunakan select_related() dan prefetch_related() secara tepat.
- Mengurangi data yang diambil dengan values(), values_list(), only(), atau pola query yang sesuai.
- Menganalisis query menggunakan QuerySet.explain()/EXPLAIN.
- Menerapkan index berdasarkan pola query dan membandingkan hasilnya.
- Menyusun performance benchmark report before vs after.

3. Alokasi Aktivitas

Tahap	Aktivitas	Estimasi
Konsep	Profiling, N+1, index, cache	25 menit
Demo	N+1 dan query count	25 menit
Lab A	Generate 10.000+ records	20 menit
Lab B	select_related / prefetch_related	35 menit
Lab C	EXPLAIN + indexing	30 menit
Tugas	Briefing Performance Optimization	10 menit

4. Prasyarat dan Environment

Prasyarat: repository Minggu 4 berjalan, migrations berfungsi, core models tersedia, Docker app + PostgreSQL aktif, dan mahasiswa dapat menggunakan Django shell.

ATURAN ENVIRONMENT
Mulai Minggu 5, jangan membuat .venv baru dan jangan meng-install Django di host. Source code tetap diedit di VS Code pada host, tetapi Django dan dependency dijalankan di container app.

Komponen	Lokasi/Peran
VS Code / source code	Host Windows; folder repository minggu-4 dilanjutkan
app	Container Django/Python
db	Container PostgreSQL
redis	Container Redis untuk eksperimen cache
manage.py	Di root project; perintah dijalankan via docker compose exec app
Git	Host; commit setelah checkpoint/tahap bermakna

Verifikasi awal — jalankan dari root repository Minggu 4:

```
docker compose ps
docker compose exec app python --version
docker compose exec app python -m django --version
docker compose exec app python manage.py check
docker compose exec app python manage.py showmigrations catalog
```

Pastikan working tree bersih sebelum eksperimen:

```
git status --short
```

KONTROL EKSPERIMEN
Gunakan environment, dataset, dan skenario query yang sama saat membandingkan before vs after. Jangan membandingkan dua kondisi yang berbeda lalu menyimpulkan optimasi berhasil.

5. Konsep Performa Database

Metrik	Makna
Query count	Jumlah query database untuk satu skenario
Execution time	Waktu eksekusi query/request
Dataset size	Jumlah record saat pengujian
Result size	Jumlah object/kolom yang dikembalikan
Improvement	Perubahan kondisi before → after

```
Improvement (%) = (before_time - after_time) / before_time × 100%
```

5.1 N+1 Query

1.apa itu N+1 Query

N+1 Query adalah masalah ketika aplikasi menjalankan satu query untuk mengambil sekumpulan data, kemudian menjalankan N query tambahan untuk mengambil data yang berelasi dengan setiap objek tersebut.

Misl: relasi User --→ Course

satu Course mempunyai satu dosen pengampu, sedangkan satu dosen dapat mengampu banyak Course.

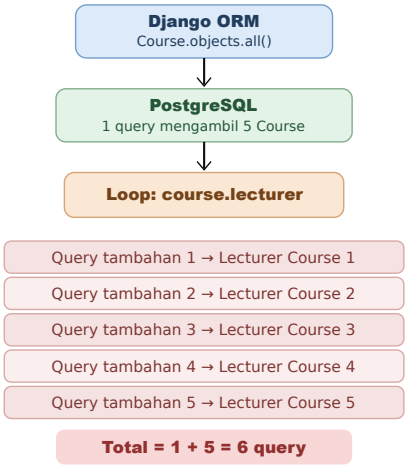
2. kode berikut dapat menyebabkan N+1 Query

```
courses = Course.objects.all()
for course in courses:
 print(course.lecturer.username)  # query tambahan
```

Jumlah Query = 1+N

dalam contoh ini N adalah jumlah Course yang diproses.

Jumlah Course	Query tanpa optimasi
1	2
5	6
20	21
100	101
1.000	1.001



Jika setiap Course memicu query lecturer, maka jumlah query bertambah mengikuti jumlah object course. Pada dataset kecil waktu dapat terlihat cepat, tetapi query count menunjukkan pola yang tidak scalable.Inilah alasan N+1 disebut masalah scalability. Semakin banyak Course, semakin banyak komunikasi dengan database.

3. Bagaimana mengatasinya?

Gunakan `select_related()` dan prefetch_related()

Teknik	Cocok untuk	Pola umum
select_related()	ForeignKey / OneToOne	SQL JOIN dalam query utama
prefetch_related()	Reverse FK / Many-to-Many	Query tambahan terkontrol + penggabungan di Python

6. Profiling — Django Debug Toolbar dan CaptureQueriesContext

Profiling adalah proses untuk memeriksa bagaimana Django menjalankan query ke database, terutama untuk mengetahui jumlah query yang digunakan dan menemukan query yang kurang efisien.

Pada praktikum ini, kita menggunakan dua tool. **CaptureQueriesContext** digunakan untuk menghitung dan membandingkan jumlah query sebelum dan sesudah optimasi secara terukur. Sementara itu, **Django Debug Toolbar** digunakan untuk melihat informasi query SQL secara langsung melalui browser saat aplikasi Django dijalankan.

Dengan kedua alat tersebut, mahasiswa dapat mengetahui apakah kode Django ORM yang dibuat sudah efisien atau masih perlu dioptimalkan.

6.1 Instalasi dependency di container

Tambahkan dependency ke requirements.txt, lalu rebuild image. Jangan pip install Django/django-redis di host.

```
Django>=5.2,<6.0

psycopg[binary]>=3.2,<4.0

django-debug-toolbar

django-redis
```

```
docker compose build app
docker compose up -d
docker compose exec app python -m django –version
```

6.2 Konfigurasi Debug Toolbar (opsional untuk inspeksi)

Di config/settings.py tambahkan debug_toolbar pada INSTALLED_APPS dan middleware sesuai kebutuhan development. Untuk Docker, INTERNAL_IPS dan network perlu disesuaikan; jika toolbar tidak muncul, gunakan CaptureQueriesContext sebagai bukti benchmark.

config/settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'users',
    'catalog',
    'debug_toolbar',
]
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'debug_toolbar.middleware.DebugToolbarMiddleware',
]
INTERNAL_IPS = ["127.0.0.1"]
```

config/urls.py

```
from django.conf import settings
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    # Django Admin
    path("admin/", admin.site.urls),
]
```

```
# Django Debug Toolbar hanya aktif saat development
if settings.DEBUG:
    urlpatterns += [
        path("__debug__/", include("debug_toolbar.urls")),
      ]
```

6.3 CaptureQueriesContext — Mengukur Jumlah Query SQL

Pada langkah ini, mahasiswa akan menggunakan CaptureQueriesContext untuk mengetahui berapa banyak query SQL yang dijalankan Django ketika mengambil data Course dari PostgreSQL.

Tujuan percobaan ini adalah membuktikan bahwa jumlah query dapat diukur secara langsung, sehingga kita tidak hanya memperkirakan apakah suatu kode Django ORM sudah efisien.

Langkah 1 — Buka Django Shell

Jalankan perintah berikut dari folder project:

```
docker compose exec app python manage.py shell
```

Langkah 2 — Jalankan kode pengukuran

```
from django.db import connection
from django.test.utils import CaptureQueriesContext
from catalog.models import Course
with CaptureQueriesContext(connection) as ctx:
    rows = list(Course.objects.all())

print("Jumlah Course:", len(rows))
print("Jumlah query SQL:", len(ctx.captured_queries))
```

Langkah 3 — Amati hasilnya

Misalnya terdapat 9 Course dan hasilnya menunjukkan 1 query. Hal tersebut berarti Django mengambil sembilan data Course menggunakan satu query SQL, bukan menjalankan sembilan query terpisah.

Langkah 4 — Buat kesimpulan

Mahasiswa menjelaskan perbedaan antara jumlah data yang diperoleh dan jumlah query SQL yang dijalankan.

CHECKPOINT 1 — Bukti Pengukuran Query

Mahasiswa menunjukkan hasil eksekusi Django Shell yang memuat jumlah Course dan jumlah query SQL. Hasil ini menjadi pengukuran awal (baseline) sebelum mempelajari dan mengoptimalkan masalah N+1 Query menggunakan `select_related()` dan `prefetch_related()`.

7. GUIDED LAB A — Generate Dataset Performa

Pada praktikum sebelumnya, kita telah membuat beberapa data Course menggunakan Django ORM. Namun, jumlah data tersebut masih sedikit sehingga belum cukup untuk menguji kinerja database ketika menangani data dalam jumlah besar.

Pada praktikum ini, kita akan **membuat 10.000 data Course secara otomatis** menggunakan program Python yang dijalankan melalui Django Management Command.

Data tersebut digunakan sebagai **data simulasi untuk menguji performa Django ORM dan PostgreSQL**, terutama untuk membandingkan jumlah query dan efisiensi pengambilan data sebelum dan sesudah optimasi.

Dalam pembuatan dataset ini, kita menggunakan dua konsep penting:

- **Repeatable:** Program dapat dijalankan berulang kali tanpa membuat data duplikat. Data yang telah dibuat pada praktikum Minggu 4 juga tetap dipertahankan.

- **bulk_create():** Fitur Django ORM untuk menyimpan banyak data sekaligus dalam beberapa kelompok (*batch*), sehingga proses pembuatan 10.000 Course lebih efisien dibandingkan menyimpan data satu per satu.

Mahasiswa tidak perlu memasukkan 10.000 Course secara manual. Cukup membuat file `generate_performance_data.py`, kemudian menjalankannya melalui terminal.

Setelah program berhasil dijalankan, mahasiswa dapat memeriksa jumlah data menggunakan Django Shell atau melihat hasilnya melalui Django Admin.

Tujuan praktikum: Mahasiswa mampu membuat dataset berukuran besar secara otomatis sebagai bahan pengujian dan optimasi query database pada tahap berikutnya.

Buat file: catalog/management/commands/generate_performance_data.py.

```
from django.core.management.base import BaseCommand
from django.db import transaction

from users.models import User
from catalog.models import Course

class Command(BaseCommand):
    help = "Buat dataset Course performa tanpa duplikasi"

    def handle(self, *args, **options):
        # 1. Cari dosen yang sudah tersedia
        lecturer = User.objects.filter(
            role="LECTURER"
        ).first()

        if lecturer is None:
            self.stderr.write(
                "Buat minimal satu User dengan role LECTURER terlebih dahulu."
            )
            return

        # 2. Ambil kode Course performa yang sudah ada
        existing = set(
            Course.objects.filter(
                code__startswith="PERF"
            ).values_list("code", flat=True)
        )

        # 3. Siapkan 10.000 Course
        rows = [
            Course(
                code=f"PERF{i:05d}",
                title=f"Performance Course {i}",
                lecturer=lecturer,
                is_active=(i % 5 != 0),
            )
            for i in range(10000)
            if f"PERF{i:05d}" not in existing
        ]

        # 4. Simpan secara batch ke PostgreSQL
        with transaction.atomic():
            Course.objects.bulk_create(
                rows,
                batch_size=500
            )

        # 5. Tampilkan hasil
        self.stdout.write(
            self.style.SUCCESS(
                f"Course baru: {len(rows)}; "
                f"total Course: {Course.objects.count()}"
            )
        )
```

Jalankan:

```
docker compose exec app python manage.py generate_performance_data
docker compose exec app python manage.py shell -c "from catalog.models import Course;
print(Course.objects.count())"
```

CATATAN CAPSTONE
Untuk tugas, jangan hanya membuat 10.000 Course. Distribusikan data pada User, Course, Enrollment, Lesson, Assignment, dan Submission agar relationship query benar-benar dapat diuji.

Selain membuat 10.000 data Course, buat juga data pada tabel-tabel lain yang saling berhubungan dalam aplikasi Simple LMS. Dataset sebagai berikut:

Model	Jumlah simulasi	Keterangan
User	55	5 dosen dan 50 mahasiswa
Course	100	Mata kuliah simulasi
Enrollment	1.000	Pendaftaran mahasiswa ke Course
Lesson	500	Rata-rata 5 materi per Course
Assignment	200	Rata-rata 2 tugas per Course
Submission	1.000	Pengumpulan tugas mahasiswa
Total	**2.855**	**Data saling berelasi**

generate_relationship_data.py

```
from datetime import timedelta

from django.core.management.base import BaseCommand, CommandError
from django.db import transaction
from django.utils import timezone

from users.models import User
from catalog.models import (
    Course,
    Enrollment,
    Lesson,
    Assignment,
    Submission,
)

class Command(BaseCommand):
    help = "Membuat dataset relasi LMS untuk pengujian performa"

    def handle(self, *args, **options):

        # ========================================
        # 1. PERIKSA FIELD MODEL
        # ========================================

        assignment_fields = {
            field.name: field
            for field in Assignment._meta.fields
        }

        submission_fields = {
            field.name: field
            for field in Submission._meta.fields
        }

        # Pastikan relasi utama tersedia
        for model, required in [
```

```
    (Assignment, {"course", "title", "deadline"}),
    (Submission, {"assignment", "student"}),
]:
    available = {
        field.name for field in model._meta.fields
    }

    missing = required - available

    if missing:
        raise CommandError(
            f"Model {model.__name__} tidak memiliki "
            f"field: {', '.join(sorted(missing))}"
        )

# Periksa field wajib lain yang belum ditangani
handled = {
    Assignment: {
        "id", "course", "title",
        "deadline", "description",
    },
    Submission: {
        "id", "assignment", "student",
        "content", "text",
    },
}

for model in (Assignment, Submission):
    for field in model._meta.fields:

        if field.name in handled[model]:
            continue

        if field.primary_key:
            continue

        if field.null or field.blank:
            continue

        if field.has_default():
            continue

        if getattr(field, "auto_now", False):
            continue

        if getattr(field, "auto_now_add", False):
            continue

        raise CommandError(
            f"Field wajib {model.__name__}."
            f"{field.name} belum ditangani generator. "
            "Periksa catalog/models.py."
        )

# ========================================
# 2. AMBIL 100 COURSE PERF
# ========================================

courses = list(
    Course.objects.filter(
        code__startswith="PERF"
    ).order_by("code")[:100]
)

if len(courses) < 100:
    raise CommandError(
        "Diperlukan minimal 100 Course PERF. "
        "Jalankan generate_performance_data dahulu."
    )

# ========================================
# 3. BUAT USER SIMULASI
# ========================================
```

```
lecturers = []
students = []

with transaction.atomic():

    for i in range(1, 6):

        username = f"perf_lecturer_{i:02d}"

        user, created = User.objects.get_or_create(
            username=username,
            defaults={
                "role": "LECTURER",
                "is_active": True,
            }
        )

        if created:
            user.set_unusable_password()
            user.save(update_fields=["password"])


        if user.role != "LECTURER":
            raise CommandError(
                f"{username} sudah ada, "
                "tetapi bukan LECTURER."
            )

        lecturers.append(user)

    for i in range(1, 51):

        username = f"perf_student_{i:03d}"

        user, created = User.objects.get_or_create(
            username=username,
            defaults={
                "role": "STUDENT",
                "is_active": True,
            }
        )

        if created:
            user.set_unusable_password()
            user.save(update_fields=["password"])

        if user.role != "STUDENT":
            raise CommandError(
                f"{username} sudah ada, "
                "tetapi bukan STUDENT."
            )

        students.append(user)

    # ====================================
    # 4. ENROLLMENT: 10 STUDENT / COURSE
    # ====================================

    existing_enrollments = set(
        Enrollment.objects.filter(
            course__in=courses
        ).values_list(
            "course_id", "student_id"
        )
    )

    enrollment_rows = []

    for index, course in enumerate(courses):


        for offset in range(10):

            student = students[
                (index * 10 + offset) % len(students)
```

```
        ]

        key = (course.id, student.id)

        if key not in existing_enrollments:

            enrollment_rows.append(
                Enrollment(
                    course=course,
                    student=student,
                )
            )

Enrollment.objects.bulk_create(
    enrollment_rows,
    batch_size=500
)

# ======================================
# 5. LESSON: 5 LESSON / COURSE
# ======================================

existing_lessons = set(
    Lesson.objects.filter(
        course__in=courses
    ).values_list(
        "course_id", "order"
    )
)

lesson_rows = []

for course in courses:

    for order in range(1, 6):

        key = (course.id, order)

        if key not in existing_lessons:

            lesson_rows.append(
                Lesson(
                    course=course,
                    title=f"Materi {order}",
                    content=(
                        f"Materi simulasi {order} "
                        f"untuk {course.code}"
                    ),
                    order=order,
                )
            )

Lesson.objects.bulk_create(
    lesson_rows,
    batch_size=500
)

# ======================================
# 6. ASSIGNMENT: 2 TUGAS / COURSE
# ======================================

existing_assignments = set(
    Assignment.objects.filter(
        course__in=courses
    ).values_list(
        "course_id", "title"
    )
)

assignment_rows = []

deadline_field = assignment_fields["deadline"]

if deadline_field.get_internal_type() == "DateField":
```

```
    deadline_value = (
        timezone.localdate() + timedelta(days=14)
    )
else:
    deadline_value = (
        timezone.now() + timedelta(days=14)
    )

for course in courses:

    for number in range(1, 3):

        title = f"PERF Tugas {number}"
        key = (course.id, title)

        if key in existing_assignments:
            continue

        values = {
            "course": course,
            "title": title,
            "deadline": deadline_value,
        }

        if "description" in assignment_fields:
            values["description"] = (
                f"Tugas simulasi untuk {course.code}"
            )

        assignment_rows.append(
            Assignment(**values)
        )

Assignment.objects.bulk_create(
    assignment_rows,
    batch_size=500
)

# ====================================
# 7. SUBMISSION: 5 / ASSIGNMENT
# ====================================

assignments = list(
    Assignment.objects.filter(
        course__in=courses,
        title__in=[
            "PERF Tugas 1",
            "PERF Tugas 2",
        ]
    ).select_related("course")
)

enrollment_map = {}

enrollments = Enrollment.objects.filter(
    course__in=courses,
    student__in=students
).values_list(
    "course_id", "student_id"
)

for course_id, student_id in enrollments:
    enrollment_map.setdefault(
        course_id, []
    ).append(student_id)

existing_submissions = set(
    Submission.objects.filter(
        assignment__in=assignments
    ).values_list(
        "assignment_id", "student_id"
    )
)
```

```
    submission_rows = []

    for assignment in assignments:

        student_ids = sorted(
            enrollment_map.get(
                assignment.course_id, []
            )
        )[:5]

        for student_id in student_ids:

            key = (assignment.id, student_id)

            if key in existing_submissions:
                continue

            values = {
                "assignment": assignment,
                "student_id": student_id,
            }

            if "content" in submission_fields:
                values["content"] = (
                    "Jawaban simulasi untuk "
                    "pengujian performa LMS."
                )

            if "text" in submission_fields:
                values["text"] = (
                    "Jawaban simulasi mahasiswa."
                )

            submission_rows.append(
                Submission(**values)
            )

    Submission.objects.bulk_create(
        submission_rows,
        batch_size=500
    )

# ========================================
# 8. TAMPILKAN HASIL
# ========================================

self.stdout.write(
    self.style.SUCCESS(
        "Dataset relasi LMS berhasil dibuat!"
    )
)

self.stdout.write(
    f"Lecturer simulasi : {len(lecturers)}"
)
self.stdout.write(
    f"Student simulasi  : {len(students)}"
)
self.stdout.write(
    f"Course digunakan  : {len(courses)}"
)
self.stdout.write(
    f"Enrollment baru   : {len(enrollment_rows)}"
)
self.stdout.write(
    f"Lesson baru       : {len(lesson_rows)}"
)
self.stdout.write(
    f"Assignment baru   : {len(assignment_rows)}"
)
self.stdout.write(
    f"Submission baru   : {len(submission_rows)}"
)
```

jalankan program, lalu lihat hasilnya di django shell atau django admin:

```
docker compose exec app python manage.py generate_relationship_data
```

Program ini akan:

1. Membuat 5 akun dosen dan 50 akun mahasiswa simulasi.
2. Menggunakan 100 Course berkode PERF yang sudah dibuat.
3. Membuat 1.000 Enrollment.
4. Membuat 500 Lesson.
5. Membuat 200 Assignment.
6. Membuat 1.000 Submission.

8. Membuat dan Mengukur Masalah N+1 Query

Pada praktikum sebelumnya, kita sudah membuat 10.000 data Course secara otomatis. Sekarang, kita akan menggunakan sebagian data tersebut untuk mempelajari masalah **N+1 Query** pada Django ORM.

N+1 Query terjadi ketika Django menjalankan satu query untuk mengambil data utama, kemudian menjalankan query tambahan untuk mengambil data yang berhubungan dengan setiap record.

Sebagai contoh, setiap Course memiliki dosen pengampu (*lecturer*). Ketika kita mengambil 100 Course beserta nama dosennya tanpa optimasi, Django dapat menjalankan **101 query SQL**, yaitu:

- 1 query untuk mengambil 100 Course.
- 100 query tambahan untuk mengambil data dosen dari masing-masing Course.

Walaupun program tetap berjalan dengan benar, terlalu banyak query dapat menurunkan performa aplikasi ketika jumlah data semakin besar.

Langkah 1 — Buka Django Shell

Jalankan perintah berikut melalui terminal pada folder project:

docker compose exec app python manage.py shell

Langkah 2 — Jalankan kode pengujian

Ketik kode berikut pada Django Shell:

```
from django.db import connection
from django.test.utils import CaptureQueriesContext
from catalog.models import Course

with CaptureQueriesContext(connection) as ctx:
    result = []
    for course in Course.objects.filter(code__startswith="PERF").order_by("code")[:100]:
        result.append((course.title, course.lecturer.username))

print("Jumlah Course:", len(result))
print("Jumlah Query SQL:", len(ctx.captured_queries))
```

Penjelasan kode:

1. `CaptureQueriesContext(connection)` digunakan untuk mencatat jumlah query SQL yang dijalankan.
2. `Course.objects.filter(code__startswith="PERF")` mengambil Course simulasi yang dibuat pada praktikum sebelumnya.
3. `[:100]` membatasi pengambilan data menjadi 100 Course.

4. `course.lecturer.username` mengambil nama dosen pengampu setiap Course. Bagian inilah yang dapat menyebabkan query tambahan.
5. `len(ctx.captured_queries)` menghitung jumlah seluruh query SQL yang dijalankan selama percobaan.

Langkah 3 — Amati hasil pengujian

Jika terdapat 100 Course, contoh hasil yang mungkin diperoleh adalah:

Jumlah Course: 100

Jumlah Query SQL: 101

Hasil tersebut menunjukkan bahwa Django menjalankan satu query untuk mengambil Course dan 100 query tambahan untuk mengambil data dosen pengampu.

Mengapa hal ini terjadi?

Karena kita belum menggunakan optimasi query. Setiap kali program mengakses `course.lecturer.username`, Django mengambil data User yang berhubungan dengan Course tersebut melalui query SQL tambahan.

Langkah 4 — Catat hasil percobaan

Mahasiswa diminta mencatat hasil pengujian pada tabel berikut.

Skenario pengujian	Jumlah Course	Jumlah query SQL	Waktu eksekusi
Course beserta Lecturer (tanpa optimasi)	100	Diisi mahasiswa	Diisi mahasiswa

Untuk mengukur waktu eksekusi, mahasiswa dapat menambahkan `import time` dan mencatat waktu sebelum serta sesudah blok pengujian.

Langkah 5 — Buat kesimpulan

Setelah menjalankan pengujian, mahasiswa menjawab pertanyaan berikut:

1. Berapa jumlah Course yang berhasil diambil?
2. Berapa jumlah query SQL yang dijalankan?
3. Mengapa jumlah query lebih banyak daripada jumlah Course?
4. Apa dampaknya jika aplikasi harus mengambil 1.000 atau 10.000 Course?

CHECKPOINT 2 — Bukti N+1 Query

Mahasiswa menunjukkan screenshot hasil pengujian Django Shell dan menjelaskan mengapa pengambilan 100 Course beserta dosen pengampunya dapat menghasilkan 101 query SQL.

Hasil pengujian ini disebut **BEFORE (sebelum optimasi)**. Pada praktikum berikutnya, mahasiswa akan menggunakan `select_related()` untuk mengurangi jumlah query SQL, kemudian membandingkan hasil sebelum dan sesudah optimasi.

9. GUIDED LAB B — Mengatasi N+1 Query dengan `select_related()`

Pada praktikum sebelumnya, kita telah mengambil 100 data Course beserta nama dosen pengampunya. Tanpa optimasi, Django dapat menjalankan 101 query SQL, yaitu satu query untuk mengambil Course dan 100 query tambahan untuk mengambil data dosen.

Pada praktikum ini, kita akan menggunakan **select_related()** untuk mengurangi jumlah query tersebut.

`select_related()` adalah fitur Django ORM yang memungkinkan data dari dua tabel yang saling berhubungan diambil sekaligus menggunakan perintah SQL `JOIN`.

Dalam aplikasi LMS kita, model `Course` memiliki hubungan `ForeignKey` dengan model `User` melalui field `lecturer`. Karena setiap Course memiliki satu dosen pengampu, hubungan ini cocok dioptimalkan menggunakan `select_related()`.

Langkah 1 — Buka Django Shell

Jalankan perintah berikut melalui terminal:

docker compose exec app python manage.py shell

Jika Django Shell dari praktikum sebelumnya masih terbuka, mahasiswa dapat langsung melanjutkan ke langkah berikutnya.

Langkah 2 — Jalankan kode setelah optimasi (AFTER)

Ketik kode berikut pada Django Shell:

```
import time

from django.db import connection
from django.test.utils import CaptureQueriesContext
from catalog.models import Course

start = time.perf_counter()

with CaptureQueriesContext(connection) as ctx:
    result = []
    courses = Course.objects.filter(code__startswith="PERF").order_by("code").select_related("lecturer")[:100]
    for course in courses:
        result.append((course.title, course.lecturer.username))

elapsed = time.perf_counter() - start

print("Jumlah Course:", len(result))
print("Jumlah Query SQL:", len(ctx.captured_queries))
print("Waktu eksekusi:", round(elapsed, 4), "detik")
```

Penjelasan kode:

1. `Course.objects.filter(code__startswith="PERF")` mengambil data Course simulasi yang telah dibuat pada Guided Lab A.
2. `[:100]` membatasi data yang diuji menjadi 100 Course, sama seperti percobaan sebelumnya.
3. `select_related("lecturer")` meminta Django mengambil data Course beserta dosen pengampunya dalam satu query SQL menggunakan `JOIN`.
4. `course.lecturer.username` mengambil nama dosen dari data yang sudah tersedia, sehingga tidak perlu menjalankan query tambahan untuk setiap Course.
5. `CaptureQueriesContext` mencatat jumlah query SQL, sedangkan `time.perf_counter()` digunakan untuk mengukur waktu eksekusi.

Langkah 3 — Amati hasil pengujian

Jika terdapat 100 Course, contoh hasil yang diharapkan adalah:

Jumlah Course: 100

Jumlah Query SQL: 1

Waktu eksekusi: 0.0123 detik

Angka waktu tersebut hanya contoh. Hasil sebenarnya dapat berbeda pada setiap komputer.

Mengapa jumlah query menjadi satu?

Karena `select_related("lecturer")` membuat Django mengambil data Course dan User yang menjadi dosen pengampu secara bersamaan menggunakan SQL `JOIN`. Dengan demikian, Django tidak perlu mengirimkan query baru setiap kali program mengakses `course.lecturer.username`.

Langkah 4 — Bandingkan hasil sebelum dan sesudah optimasi

Isilah tabel berikut berdasarkan hasil percobaan pada Subbab 8 dan Subbab 9.

Skenario	Sebelum optimasi (BEFORE)	Sesudah optimasi (AFTER)
Jumlah Course	100	100
Jumlah query SQL	Diisi mahasiswa	Diisi mahasiswa
Waktu eksekusi	Diisi mahasiswa	Diisi mahasiswa
Teknik pengambilan data	Query biasa	`select_related()`

Dari tabel tersebut, mahasiswa dapat melihat apakah jumlah query SQL berkurang setelah menggunakan `select_related()`.

Langkah 5 — Buat kesimpulan

Setelah membandingkan hasil pengujian, mahasiswa menjawab pertanyaan berikut:

1. Berapa jumlah query sebelum menggunakan `select_related()`?
2. Berapa jumlah query setelah menggunakan `select_related()`?
3. Mengapa jumlah query SQL dapat berkurang?
4. Mengapa `select_related()` cocok digunakan pada hubungan Course dengan Lecturer?

CHECKPOINT 2 — Memahami Fungsi select_related()

Mahasiswa harus dapat menunjukkan hasil pengujian sebelum dan sesudah optimasi serta menjelaskan alasan penggunaan `select_related()`.

`select_related()` cocok digunakan karena `lecturer` merupakan `ForeignKey` pada model `Course`, sehingga setiap Course mengacu pada satu User sebagai dosen pengampu. Django dapat mengambil kedua data tersebut sekaligus melalui SQL `JOIN`.

Kesimpulan Praktikum

Dengan menggunakan `select_related()`, jumlah query untuk mengambil 100 Course beserta dosen pengampunya dapat berkurang dari 101 query menjadi hanya 1 query. Hal ini menunjukkan bahwa optimasi Django ORM dapat mengurangi jumlah akses ke database dan membantu meningkatkan efisiensi aplikasi.

10. GUIDED LAB C — Mengatasi N+1 Query dengan `prefetch_related()`

Pada praktikum sebelumnya, kita menggunakan `select_related()` untuk mengambil data Course beserta dosen pengampunya.

Sekarang, kita akan menggunakan **prefetch_related()** untuk mengambil data Course beserta materi pembelajarannya (*Lesson*).

Satu Course dapat memiliki banyak Lesson. Jika data Lesson diambil satu per satu untuk setiap Course, Django akan menjalankan banyak query SQL. Masalah ini disebut **N+1 Query**.

Dengan `prefetch_related("lessons")`, Django dapat mengambil seluruh Lesson yang dibutuhkan melalui satu query tambahan, sehingga jumlah query menjadi lebih sedikit.

Langkah 1 — Buka Django Shell

Jalankan:

```
docker compose exec app python manage.py shell
```

Langkah 2 — Pengujian BEFORE (tanpa optimasi)

```
from django.db import connection
from django.test.utils import CaptureQueriesContext
from catalog.models import Course

with CaptureQueriesContext(connection) as ctx:
    courses = Course.objects.filter(code__startswith="PERF").order_by("code")[:50]
    for course in courses:
        titles = [lesson.title for lesson in course.lessons.all()]

print("Jumlah query BEFORE:", len(ctx.captured_queries))
```

Kode tersebut mengambil 50 Course beserta Lesson-nya tanpa optimasi.

Hasil yang diharapkan: 51 query, yaitu 1 query untuk Course dan 50 query tambahan untuk Lesson.

Langkah 3 — Pengujian AFTER (dengan optimasi)

```
with CaptureQueriesContext(connection) as ctx:
    courses = Course.objects.filter(code__startswith="PERF").order_by("code").prefetch_related("lessons")[:50]
    for course in courses:
        titles = [lesson.title for lesson in course.lessons.all()]

print("Jumlah query AFTER:", len(ctx.captured_queries))
```

Hasil yang diharapkan: 2 query, yaitu 1 query untuk Course dan 1 query untuk seluruh Lesson yang berhubungan dengan Course tersebut.

Langkah 4 — Bandingkan hasil

Pengujian	BEFORE	AFTER
Jumlah Course	50	50
Jumlah query SQL	Diisi mahasiswa	Diisi mahasiswa
Teknik	Query biasa	`prefetch_related()`

CHECKPOINT 3 — Memahami prefetch_related()

Mahasiswa menunjukkan hasil pengujian dan menjelaskan mengapa jumlah query berkurang.

Kesimpulan: `prefetch_related()` cocok digunakan pada hubungan **satu-ke-banyak (One-to-Many)**, seperti Course dengan Lesson. Django mengambil Course dan Lesson melalui dua query terpisah, kemudian menghubungkan hasilnya di Python. Dengan cara ini, Django tidak perlu menjalankan query tambahan untuk setiap Course.

11. GUIDED LAB D — Mengenal Teknik Optimasi Query Lain

Selain `select_related()` dan `prefetch_related()`, Django ORM menyediakan beberapa fungsi untuk mengambil dan mengolah data secara lebih efisien.
Pada praktikum ini, kita akan mencoba empat teknik menggunakan database LMS yang sudah dibuat sebelumnya.
Langkah 1 — Buka Django Shell
Jalankan perintah:
docker compose exec app python manage.py shell
Kemudian impor model dan fungsi yang diperlukan:

```
from catalog.models import Course, Enrollment
from django.db.models import Count
```

Percobaan A — exists(): Memeriksa keberadaan data
Misalnya kita ingin mengetahui apakah Course `PERF00001` memiliki mahasiswa yang terdaftar.

```
course = Course.objects.get(code="PERF00001")

ada_mahasiswa = Enrollment.objects.filter(course=course).exists()

print("Ada mahasiswa terdaftar:", ada_mahasiswa)
```

Penjelasan: `exists()` digunakan ketika kita hanya ingin mengetahui apakah data tersedia atau tidak. Hasilnya berupa `True` atau `False`, tanpa perlu mengambil seluruh objek Enrollment.

Percobaan B — count(): Menghitung jumlah data

```
jumlah_course = Course.objects.filter(code__startswith="PERF").count()

print("Jumlah Course simulasi:", jumlah_course)
```

Penjelasan: `count()` meminta database menghitung jumlah data. Kita tidak perlu mengambil seluruh objek Course ke dalam memori Python hanya untuk menghitungnya.

Percobaan C — values(): Mengambil kolom tertentu

```
courses = Course.objects.filter(code__startswith="PERF").values("code", "title")[:5]

for course in courses:
    print(course)
```

Penjelasan: `values()` digunakan untuk mengambil kolom tertentu saja. Pada contoh ini, Django hanya mengambil `code` dan `title`, bukan seluruh field Course.

Percobaan D — annotate(): Menghitung jumlah mahasiswa setiap Course

```
courses =
Course.objects.filter(code__startswith="PERF").annotate(jumlah_mahasiswa=Count("enrollments")).order_by("code")[:5]

for course in courses:
    print(course.code, course.jumlah_mahasiswa)
```

Penjelasan: `annotate()` menambahkan hasil perhitungan pada setiap Course. Pada contoh ini, kita menghitung jumlah Enrollment untuk masing-masing Course menggunakan fungsi `Count()`.

CHECKPOINT 4 — Memahami Optimasi Query
Mahasiswa menjalankan keempat percobaan, kemudian menunjukkan hasilnya melalui screenshot Django Shell. Selanjutnya, mahasiswa menjelaskan fungsi `exists()`, `count()`, `values()`, dan `annotate()` serta memberikan contoh kapan masing-masing teknik digunakan.

Kesimpulan: Optimasi query tidak selalu berarti mengurangi jumlah query SQL. Optimasi juga dapat dilakukan dengan mengambil hanya data yang diperlukan atau meminta database melakukan perhitungan, sehingga pengolahan data menjadi lebih efisien.

12. GUIDED LAB E — Memeriksa Query dan Membuat Database Index

Pada praktikum ini, kita akan mempelajari cara PostgreSQL mencari data menggunakan `QuerySet.explain()` dan cara menambahkan **database index**.
Index dapat diibaratkan seperti daftar isi pada buku. Dengan index, database dapat menemukan data tertentu tanpa harus membaca seluruh baris tabel. Namun, tidak semua query akan menjadi lebih cepat setelah diberi index.

Langkah 1 — Memeriksa cara PostgreSQL mencari data
Buka Django Shell:
docker compose exec app python manage.py shell
Jalankan kode berikut:
from catalog.models import Course

qs = Course.objects.filter(code="PERF05000")

print(qs.explain())
Penjelasan: `explain()` menampilkan rencana PostgreSQL dalam menjalankan query. Misalnya, apakah database menggunakan index (*Index Scan*) atau membaca tabel (*Seq Scan*).
Field `code` sudah menggunakan `unique=True`, sehingga PostgreSQL biasanya telah membuat index unik untuk field tersebut.

Langkah 2 — Tambahkan index baru
Buka file `catalog/models.py`, lalu cari class `Course`.
Pada bagian `class Meta` yang sudah ada, tambahkan `indexes` seperti berikut:

```
class Meta:
    ordering = ["code"]
    indexes = [
        models.Index(fields=["is_active", "title"]),
    ]
```

Penting: Jangan membuat class `Course` atau `class Meta` kedua. Cukup tambahkan `indexes` pada `class Meta` yang sudah ada.
Index tersebut dibuat untuk membantu query yang mencari Course berdasarkan status aktif dan judul.

Langkah 3 — Terapkan perubahan ke PostgreSQL
Jalankan perintah berikut:
docker compose exec app python manage.py makemigrations
docker compose exec app python manage.py migrate

Perintah `makemigrations` membuat file perubahan struktur database, sedangkan `migrate` menerapkan perubahan tersebut ke PostgreSQL.

Langkah 4 — Uji query setelah penambahan index
Buka kembali Django Shell:
docker compose exec app python manage.py shell
Kemudian jalankan:
from catalog.models import Course

qs = Course.objects.filter(is_active=True,title="Performance Course 5000")

print(qs.explain())

Perhatikan apakah PostgreSQL menggunakan index baru. Untuk membuktikan manfaatnya, bandingkan juga rencana query dan waktu eksekusi dengan hasil sebelum index dibuat.

Catatan: PostgreSQL tidak selalu menggunakan index. Pemilihannya bergantung pada jumlah data, kondisi tabel, dan jenis query. Index juga memerlukan ruang penyimpanan tambahan serta dapat menambah biaya operasi INSERT, UPDATE, dan DELETE.

CHECKPOINT 4 — Pengujian Database Index
Mahasiswa menunjukkan bukti:

1. Hasil `explain()` sebelum penambahan index.
2. Kode index pada model `Course`.
3. Hasil migrasi database.
4. Hasil `explain()` setelah penambahan index.

Kesimpulan: Database index dapat membantu mempercepat pencarian data tertentu. Namun, keberhasilan membuat index belum tentu berarti query menjadi lebih cepat. Manfaatnya perlu dibuktikan melalui hasil pengujian.

13. Git Checkpoint dan Workflow

Gunakan Git untuk menyimpan checkpoint eksperimen. Contoh commit yang jelas:

```
git status --short
git add .
git commit -m "feat: add performance dataset generator"

git add .
git commit -m "perf: optimize course lecturer query"

git push
```

ATURAN REPOSITORY
Jangan commit .env berisi secret, database dump yang tidak diperlukan, cache, atau file hasil eksperimen berukuran besar. README harus menjelaskan cara membuat dataset dan mengulang benchmark.

14. TUGAS MINGGU 5 — Optimasi Performa Database

A. Tujuan Tugas

Mahasiswa mampu menemukan query yang kurang efisien, memperbaikinya menggunakan Django ORM, dan membuktikan hasil perbaikannya melalui pengujian.

Tugas dikerjakan menggunakan **project Simple LMS dari Minggu 4**. Mahasiswa tidak perlu membuat project baru.

B. Langkah Pengerjaan

1. **Siapkan data pengujian.** Gunakan minimal 10.000 data Course yang telah dibuat pada praktikum.
2. **Uji N+1 Query pada Course dan Lecturer.** Bandingkan jumlah query sebelum dan sesudah menggunakan `select_related()`.
3. **Uji N+1 Query pada Course dan Lesson.** Bandingkan jumlah query sebelum dan sesudah menggunakan `prefetch_related()`.
4. **Uji database index.** Tambahkan satu index pada model `Course`, jalankan migrasi, kemudian periksa hasilnya menggunakan `explain()`.
5. **Jalankan pengujian aplikasi.** Pastikan fitur dan automated test yang sudah dibuat tetap berjalan dengan baik.
6. **Catat hasil percobaan.** Buat tabel perbandingan dan jelaskan hasilnya secara singkat.

C. Tabel Hasil Pengujian

Mahasiswa mengisi tabel berikut berdasarkan hasil percobaan.

Percobaan	Query BEFORE	Query AFTER	Kesimpulan
Course + Lecturer	...	...	...
Course + Lesson	...	...	...
Database Index	Hasil EXPLAIN sebelum	Hasil EXPLAIN sesudah	...

Untuk pengujian database index, mahasiswa juga mencatat apakah waktu eksekusi berubah. Jumlah query saja tidak cukup untuk menilai manfaat index.

D. Yang Harus Dikumpulkan

Mahasiswa mengumpulkan:

1. **Link repository GitHub** berisi kode project Simple LMS yang telah diperbarui.
2. **Laporan singkat (PDF)** berisi tabel hasil pengujian, screenshot hasil Django Shell, dan kesimpulan.
3. **Bukti pengujian aplikasi**, misalnya screenshot hasil `python manage.py test`.

E. Format Laporan Singkat

LAPORAN OPTIMASI DATABASE — MINGGU 5

Nama MHS:

NIM :

Kelas:

Repository GitHub: ……..

1. Data Pengujian

Jumlah Course: ….

Jumlah Lecturer, Student, Lesson, Enrollment, Assignment, dan Submission: ‘…….

2. Hasil Pengujian

Isi tabel perbandingan BEFORE dan AFTER serta lampirkan screenshot (point **C**).

3. Kesimpulan

Jelaskan optimasi apa yang dilakukan, apakah jumlah query berkurang, dan apa manfaatnya bagi aplikasi LMS.

F. Penilaian Tugas

Komponen	Bobot
Pengujian dan perbaikan N+1 Query	40%
Pengujian database index	25%
Tabel hasil dan bukti pengujian	25%
Kerapian kode dan laporan	10%
Total	**100%**

Catatan: Nilai diberikan berdasarkan ketepatan kode, hasil pengujian, dan kemampuan menjelaskan percobaan. Mahasiswa tidak boleh mengarang hasil benchmark.